Que son y para que sirven las peticiones HTTP:

HTTP DEFINICIÓN:

HTTP es un protocolo. De hecho, el acrónimo significa Protocolo de Transferencia de Hipertexto. Este protocolo rige la estructura y el lenguaje de las peticiones y respuestas que tienen lugar entre clientes y servidores. Los clientes suelen ser los navegadores web, pero pueden presentarse de muchas formas, como los robots de los motores de búsqueda.

Cuando escribes una URL como:

https://www.google.com

tu navegador envía una petición HTTP al servidor de Google. El servidor procesa la solicitud y responde con una página web, imágenes, datos u otro contenido.

¿Para qué sirven?

Las peticiones HTTP se usan para:

- Obtener páginas web.
- Consultar datos de una API.
- Enviar formularios.
- Iniciar sesión en aplicaciones.
- Subir archivos.
- Actualizar o eliminar información en una base de datos.

Piensa en una petición HTTP como si tu navegador se conectara al servidor y le pidiera un recurso específico o le enviara datos. Hay varios tipos de métodos de petición HTTP, que modifican completamente el tipo de respuesta que obtienes del servidor. Los más comunes son:

Método	Uso
GET	Obtener información
POST	Crear o enviar datos

PUT	Actualizar un recurso completo
PATCH	Actualizar parcialmente un recurso
DELETE	Eliminar un recurso

Presentar una solicitud HTTP implica enviar un mensaje al servidor receptor en un formato específico. El servidor devuelve una respuesta y el cliente realiza la acción correspondiente. Por ejemplo, puede cargar recursos o redirigirte a otra página.

Componentes de una petición HTTP

Una petición normalmente contiene:

1. **Método** (GET, POST, PUT, etc.)
2. **URL** del recurso.
3. **Cabeceras (Headers)** con información adicional.
4. **Body (cuerpo)** con datos, en caso de ser necesario.

JWT

Un **JWT (JSON Web Token)**es una cadena compacta y **segura para URLs** que representa un conjunto de información (normalmente sobre un usuario o sesión) de forma que puede transmitirse entre sistemas. A menudo lo verás como un valor largo que empieza con algo como eyJ..., enviado en una cabecera HTTP como Authorization: Bearer \u003ctoken\u003e.

Esta información puede ser verificada y confiable porque está firmada digitalmente. Los JWT pueden ser firmados usando un secreto (con el algoritmo HMAC) o un par de claves públicas/privadas usando RSA o ECDSA.

*RSA y ECDSA son dos algoritmos criptográficos de clave asimétrica que se utilizan ampliamente para asegurar datos, autenticar identidades y crear firmas digitales. [1, 2]

Ambos emplean un par de claves matemáticas: una clave privada (mantenida en secreto para firmar o descifrar) y una clave pública (compartida libremente para verificar o cifrar). Sin embargo, funcionan con bases matemáticas muy diferentes. *

Estructura de un JWT

Un JWT tiene tres partes separadas por puntos (.):xxxxx.yyyyy.zzzzz

```
xxxxx.yyyyy.zzzzz
```

1. Header

Indica el algoritmo utilizado:

```json
{
  "alg": "HS256",
  "typ": "JWT"
}
```

2. Payload

Contiene la información del usuario:

```json
{
  "id": 1,
  "nombre": "Zaida",
  "rol": "admin"
}
```

3. Signature

Es la firma que garantiza que el token no ha sido modificado.

¿Cómo usar JWT en Express?

1. Instalar dependencias

```
npm install express jsonwebtoken bcryptjs
```

2. Crear el servidor

```
const express = require("express");
const jwt = require("jsonwebtoken");

const app = express();
app.use(express.json());

const SECRET_KEY = "mi_clave_secreta";
```

3. Generar un token al iniciar sesión

```
app.post("/login", (req, res) => {
    const { usuario, password } = req.body;

    // Simulación de validación
    if (usuario === "zaida" && password === "123") {

        const token = jwt.sign(
            {
                usuario: usuario,
                rol: "admin"
            },
            SECRET_KEY,
            { expiresIn: "1h" }
        );

        return res.json({ token });
    }

    res.status(401).json({
        mensaje: "Credenciales incorrectas"
    });
});
```

4. Crear un middleware para validar el JWT

```
function verificarToken(req, res, next) {

    const authHeader = req.headers.authorization;

    if (!authHeader) {
        return res.status(401).json({
            mensaje: "Token requerido"
        });
    }

    const token = authHeader.split(" ")[1];

    try {
        const decoded = jwt.verify(token, SECRET_KEY);

        req.usuario = decoded;

        next();

    } catch (error) {
        return res.status(403).json({
            mensaje: "Token inválido"
        });
    }
}
```

5. Proteger rutas

```
app.get("/perfil", verificarToken, (req, res) => {
    res.json({
        mensaje: "Acceso autorizado",
        usuario: req.usuario
    });
});
```

6. Enviar el token desde el cliente

La petición HTTP debe incluir el encabezado:

```
Authorization: Bearer eyJhbGciOiJIUzI1Ni...
```

```
fetch("http://localhost:3000/perfil", {
    headers: {
        Authorization: `Bearer ${token}`
    }
});
```

¿Por qué utilizar JWT?

Las ventajas de utilizar JWT incluyen las siguientes:

Son livianos y fáciles de usar para aplicaciones cliente: por ejemplo, aplicaciones móviles.

Son autónomos, lo que significa que el servidor Liberty JVM puede consumir el token directamente y utilizar una reclamación del token como identidad para ejecutar la solicitud.

Pueden estar firmados simétricamente mediante un secreto compartido mediante el algoritmo HMAC o asimétricamente mediante una clave privada.

Tienen un mecanismo de caducidad incorporado.

Se pueden ampliar para que contengan declaraciones personalizadas.

Son ampliamente adoptados por diferentes soluciones de inicio de sesión único y estándares bien conocidos, como Cómo funciona:OpenID Conectar .

¿Qué es CRUD?

CRUD es un acrónimo que representa las 4 operaciones básicas que se realizan sobre los datos en una aplicación:

- **C** = Create (Crear)
- **R** = Read (Leer)
- **U** = Update (Actualizar)
- **D** = Delete (Eliminar)

Por ejemplo, en una aplicación de empleados:

- Crear un empleado → **Create**

- Consultar empleados → **Read**
- Modificar datos de un empleado → **Update**
- Eliminar un empleado → **Delete**

CRUD y HTTP

En las APIs REST, cada operación CRUD suele asociarse con un método HTTP:

CRUD	Método HTTP	Ejemplo
Create	POST	Crear un usuario
Read	GET	Obtener usuarios
Update	PUT/PATCH	Actualizar usuario
Delete	DELETE	Eliminar usuario

¿Qué es el middleware?

El middleware es un software que permite la comunicación entre diferentes aplicaciones. Brinda funcionalidad para conectar las aplicaciones de manera inteligente y eficiente, de forma que se pueda innovar más rápido. El *middleware* actúa como un puente entre tecnologías, herramientas y bases de datos diversas para que pueda integrarlas sin dificultad en un único sistema. Este sistema único provee un servicio unificado a sus usuarios. Por ejemplo, una aplicación frontend de Windows envía y recibe datos desde un servidor backend de Linux, pero los usuarios de la aplicación no están al tanto de la diferencia.
Proporciona servicios y funciones comunes, como Single Sign-On (SSO) o la gestión de la interfaz de programación de aplicaciones (API). Los desarrolladores cuentan con que el middleware les ofrecerá uniformidad y sencillez en las integraciones de los elementos de las aplicaciones. Esto los libera para diseñar las funciones más importantes de las aplicaciones en vez de invertir tiempo en conectarlas a diferentes extremos y entornos, lo que incluye sistemas heredados.

El middleware actual es la base tecnológica de las arquitecturas modernas diseñadas para la nube. En el caso de las empresas con entornos de contenedores y multicloud, el middleware puede hacer que sea más rentable desarrollar y ejecutar las aplicaciones según sea necesario.

En esencia, el middleware funciona como una capa de comunicación que gestiona tareas como:

- **Mensajería:** Gestionar colas y temas para que las aplicaciones puedan intercambiar información de forma asíncrona, incluso si un sistema está desconectado
- **Gestión de API:** Exponer API estandarizadas para que las aplicaciones puedan consumir servicios de forma consistente y reutilizable, una capacidad común en entornos de plataforma como servicio (PaaS)
- **Seguridad e identidad:** Aplicación de la autenticación, autorización y cifrado
- **Acceso a datos:** Optimización de consultas de bases de datos, soporte para integración de datos en múltiples fuentes y ayuda para mantener la coherencia en sistemas distribuidos
- **Transformación y enrutamiento de datos:** Convertir la información al formato correcto para que los sistemas se entiendan entre sí y luego dirigir las solicitudes al destino correcto
- **Manejo y monitoreo de errores:** Señalización de problemas y envío de alertas antes de que interrumpan los flujos de trabajo
- **Gestión de sesiones:** Mantener las sesiones de usuario, lo cual es especialmente importante para aplicaciones web y móviles

Como se comunica node con diferentes archivos:

En **Node.js**, los archivos se comunican entre sí mediante el sistema de **módulos** usando `require` (CommonJS) o `import/export` (ES Modules).

Opción 1: Usando `require`

archivo: `saludos.js`

```
function saludar(nombre) {
    return `Hola ${nombre}`;
}

module.exports = saludar;
```

archivo: `app.js`

```
const saludar = require("./saludos");

console.log(saludar("Zaida"));
```

Exportar varios elementos

usuarios.js

```
const usuarios = ["Ana", "Juan"];

function obtenerUsuarios() {
    return usuarios;
}

module.exports = {
    usuarios,
    obtenerUsuarios
};
```

app.js

```
const userController = require("./usuarios");

console.log(userController.usuarios);
console.log(userController.obtenerUsuarios());
```

¿Qué son las bases de datos no relacionales?

Las **bases de datos no relacionales** (también llamadas **NoSQL**) son sistemas de almacenamiento de datos que **no utilizan tablas relacionadas mediante claves foráneas**, como ocurre en bases de datos relacionales como MySQL, PostgreSQL o SQL Server.

En lugar de tablas, pueden almacenar información en:

- Documentos (JSON)
- Clave-valor
- Grafos
- Columnas

Las bases de datos NoSQL priorizan la flexibilidad, la escalabilidad y el rendimiento, lo que las hace adecuadas para grandes volúmenes de datos distribuidos, semiestructurados o que cambian rápidamente.

Entre los tipos comunes de bases de datos NoSQL se incluyen:

- **Bases de datos de clave-valor:** almacene los datos como pares simples de claves y valores, de forma similar a un diccionario. Normalmente se usan para cosas como el almacenamiento en caché, el almacenamiento de sesión y las búsquedas rápidas, donde la estructura de datos es sencilla.
- **Bases de datos de documentos:** almacene datos como documentos, a menudo en formatos como JSON. Este enfoque funciona bien cuando los datos cambian con el tiempo, como perfiles de usuario, catálogos de productos o contenido almacenado por una aplicación.
- **Bases de datos de columnas anchas:** organice los datos en filas con columnas flexibles que pueden variar de una entrada a la siguiente. A menudo se usan para controlar grandes volúmenes de datos y cargas de trabajo que requieren un alto rendimiento de escritura.
- **Bases de datos Graph:** almacene datos como nodos conectados y relaciones. Entre los ejemplos comunes se incluyen redes sociales, sistemas de recomendaciones y análisis de fraudes.

Diferencias entre las bases de datos NoSQL y SQL

Las bases de datos NoSQL y SQL están diseñadas para satisfacer diferentes necesidades de datos y aplicaciones. En el resumen siguiente se resaltan las diferencias clave de un vistazo.

Modelo de datos

- **Las bases de datos SQL** almacenan datos en tablas estructuradas con esquemas predefinidos.
- **Las bases de datos NoSQL** admiten modelos de datos flexibles, lo que permite que las estructuras de datos cambien más fácilmente con el tiempo.

Diseño de esquemas

- **Las bases de datos SQL** requieren que los esquemas se definan de antemano, lo que ayuda a aplicar la coherencia.
- **Las bases de datos NoSQL** permiten que los esquemas evolucionen a medida que aumentan las aplicaciones, reduciendo la necesidad de cambios estructurales frecuentes.

Escalabilidad

- **Las bases de datos SQL** normalmente se escalan agregando más recursos a un único servidor.
- **Las bases de datos NoSQL** suelen diseñarse para escalarse en varios servidores, lo que admite cargas de trabajo más grandes y distribuidas.

Coherencia y transacciones

- **Las bases de datos SQL** enfatizan la coherencia fuerte y admiten transacciones complejas.
- **Las bases de datos NoSQL** equilibran la coherencia, la disponibilidad y el rendimiento en función del tipo de base de datos y el caso de uso.

Casos de uso típicos

- **Las bases de datos SQL** se suelen usar para sistemas transaccionales en los que la precisión de los datos es fundamental.
- **Las bases de datos NoSQL** se usan normalmente para aplicaciones que priorizan la escalabilidad, la velocidad y las estructuras de datos flexibles.

Cómo afectan estas diferencias a las aplicaciones reales

En la práctica, las bases de datos SQL, incluidos los sistemas ampliamente usados como PostgreSQL, son una opción segura cuando las relaciones de datos son estables y la integridad transaccional es esencial. Muchos sistemas modernos usan bases de datos SQL y NoSQL juntas, seleccionando el enfoque adecuado para cada carga de trabajo en función de la estructura y la escala de los datos.

MONGO DB

MongoDB es un sistema de base de datos NoSQL orientada a documentos diseñado para almacenar grandes volúmenes de datos con alta flexibilidad y escalabilidad. A diferencia de las bases de datos relacionales tradicionales (como MySQL o PostgreSQL) que utilizan tablas, filas y columnas, la página oficial de MongoDB explica que este sistema organiza la información en **documentos similares a JSON** (técnicamente almacenados en formato BSON). [1, 2, 3, 4]

Características principales

- **Esquema dinámico:** cada documento puede tener un número variable de campos con estructuras diferentes entre sí.
- **Escalabilidad horizontal:** distribuye los datos eficientemente en múltiples servidores mediante una técnica llamada *sharding*.
- **Alta disponibilidad:** utiliza "conjuntos de réplicas" para copiar los datos en varios nodos y evitar pérdidas por fallos de hardware.
- **Consultas potentes:** soporta indexación avanzada, búsquedas de texto y agregaciones complejas en tiempo real. [1, 2, 3]

Estructura de datos vs. Bases de datos Relacionales

Para entender su funcionamiento, es útil comparar sus componentes básicos con los del modelo clásico de tablas:

Concepto Relacional (SQL)	**Concepto en MongoDB (NoSQL)**	**Descripción**
Base de datos	Base de datos	Contenedor físico para las colecciones.
Tabla	Colección	Agrupación de documentos (equivalente a una tabla).
Fila / Registro	Documento	El registro individual de datos en formato JSON/BSON.
Columna	Campo	Par clave-valor dentro de un documento.

¿Para qué se utiliza?

Según los expertos en tecnología de corporaciones como IBM, MongoDB es ideal para escenarios donde la velocidad de desarrollo y la evolución de los datos son críticas: [1]

- **Aplicaciones móviles y web:** gestión de perfiles de usuario, carritos de compra y sesiones.

- **Plataformas de Big Data:** almacenamiento de flujos de datos masivos que no siguen un formato estricto.
- **Sistemas de gestión de contenidos (CMS):** plataformas de blogs o e-commerce donde cada producto o artículo tiene atributos únicos.
- **Catálogos de productos:** almacenamiento de inventarios con especificaciones muy variadas y cambiante

Mongoose es una biblioteca de Modelado de Datos de Objetos (ODM) para MongoDB y Node.js. Actúa como un intermediario que traduce los datos de la base de datos en objetos de JavaScript, permitiendo aplicar una estructura estricta (esquema) a una base de datos que por naturaleza no la tiene.

¿Por qué usar Mongoose?

- **Esquemas obligatorios:** Define qué campos se permiten, sus tipos de datos y si son requeridos.
- **Validación integrada:** Comprueba que los datos cumplan con reglas específicas antes de guardarlos.
- **Middlewares / Hooks:** Ejecuta funciones automáticas antes o después de ciertas acciones (como encriptar una contraseña antes de guardar).
- **Consultas simplificadas:** Reduce la cantidad de código necesario para buscar, actualizar o eliminar registros.

Ejemplos Prácticos de Uso

Para usar Mongoose, primero debes instalarlo en tu proyecto de Node.js ejecutando npm install mongoose en tu terminal.

1. Conexión a la Base de Datos

Este fragmento de código establece la conexión inicial con tu servidor local de MongoDB:

```javascript
const mongoose = require('mongoose');

// Conectar a la base de datos llamada 'mi_tienda'
mongoose.connect('mongodb://localhost:2017/mi_tienda')
  .then(() => console.log('Conexión exitosa a MongoDB'))
  .catch(err => console.error('Error al conectar a la base de datos:', err));
```

2. Definición de un Esquema y Modelo

El **Esquema** define la estructura del documento y el **Modelo** es la clase constructora que te permite interactuar con la colección.

```javascript
const { Schema, model } = mongoose;

// 1. Definir el esquema con validaciones básicas
const productoSchema = new Schema({
  nombre: { type: String, required: true },
  precio: { type: Number, required: true, min: 0 },
  categoria: String,
  disponible: { type: Boolean, default: true },
  fechaCreacion: { type: Date, default: Date.now }
});

// 2. Crear el modelo basado en el esquema
// Nota: Mongoose creará automáticamente la colección en plural ('productos')
const Producto = model('Producto', productoSchema);
```

3. Operaciones CRUD Básicas

Una vez definido el modelo, puedes realizar operaciones de creación, lectura, actualización y eliminación de forma asíncrona:

Crear un documento (Insertar)

javascript

```javascript
async function crearProducto() {
  try {
    const nuevoProducto = new Producto({
      nombre: 'Laptop Gamer',
      precio: 1299.99,
      categoria: 'Electrónica'
    });

    const resultado = await nuevoProducto.save();
    console.log('Producto guardado:', resultado);
  } catch (error) {
    console.error('Error al guardar:', error.message);
  }
}
```

Buscar documentos (Leer)

javascript

```javascript
async function buscarProductos() {
  try {
    // Buscar todos los productos de la categoría 'Electrónica'
    const electronicos = await Producto.find({ categoria: 'Electrónica' });
    console.log('Productos encontrados:', electronicos);
  } catch (error) {
    console.error('Error en la búsqueda:', error);
  }
}
```

Actualizar un documento

javascript

```javascript
async function actualizarPrecio(id, nuevoPrecio) {
  try {
    const productoActualizado = await Producto.findByIdAndUpdate(
      id,
      { precio: nuevoPrecio },
      { new: true } // Retorna el documento ya modificado
    );
    console.log('Producto actualizado:', productoActualizado);
  } catch (error) {
    console.error('Error al actualizar:', error);
  }
}
```